

PostgreSQL Index Strategy — bSecure

Table of Contents

1. [Overview & Principles](#)
 2. [Index Types Used](#)
 3. [Indexes by Table](#)
 4. [Partial Indexes](#)
 5. [Expression Indexes](#)
 6. [Covering Indexes \(INCLUDE Clause\)](#)
 7. [GiST Spatial Indexes](#)
 8. [Composite Index Design Decisions](#)
 9. [EXPLAIN ANALYZE Examples](#)
 10. [Index Maintenance](#)
 11. [Index Bloat & Monitoring](#)
 12. [Full Index Creation Script](#)
-

1. Overview & Principles

Indexes are the single most impactful performance lever for a booking platform with real-time location tracking. Every query that touches a table with more than ~10k rows should be backed by a deliberate index strategy. The rules below are non-negotiable.

The Six Rules

#	Rule	Rationale
1	Every foreign key must be indexed	PostgreSQL does not auto-index FKs. Without an index, a DELETE on a parent row causes a sequential scan on every child table.
2	Never index a low-cardinality column alone	A B-tree index on status (5 values) or is_available (boolean) will be ignored by the planner for large tables — the bitmap scan cost exceeds the seq scan

#	Rule	Rationale
		cost. Combine with a high-cardinality column or use a partial index.
3	Prefer partial indexes over full indexes	A partial index on <code>WHERE status = 'pending'</code> is 20× smaller than a full index and fits entirely in <code>shared_buffers</code> . The planner will always prefer it for matching queries.
4	Use GiST for all geography/geometry columns	PostGIS GiST supports bounding-box overlap (<code>&&</code>), containment, and KNN distance (<code><-></code>) queries. B-tree cannot index spatial data.
5	Use covering indexes (INCLUDE) to achieve index-only scans	When all columns needed by a query are in the index, PostgreSQL can return results without touching the heap. This eliminates random I/O entirely for hot read paths.
6	Always create indexes with CONCURRENTLY in production	<code>CREATE INDEX</code> without <code>CONCURRENTLY</code> takes an <code>AccessShareLock</code> that blocks all writes. <code>CONCURRENTLY</code> takes multiple passes and only holds a weak lock, allowing normal read/write traffic throughout.

2. Index Types Used

B-tree (default)

The default index type. Supports equality (`=`), range (`<`, `>`, `BETWEEN`), and `ORDER BY / LIMIT` pushdown. Used for virtually all scalar columns.

```
CREATE INDEX ON bookings_booking (user_id, created_at DESC);
```

GiST (Generalized Search Tree) — PostGIS

Used exclusively for `GEOGRAPHY` and `GEOMETRY` columns. Supports bounding-box overlap (`&&`), containment (`@>`), and KNN distance (`<->`). Also supports `EXCLUDE` constraints on overlapping ranges.

```
CREATE INDEX ON guards_profile USING gist (current_location);
```

GIN (Generalized Inverted Index) — Full-Text Search

Used for tsvector columns. A GIN index maps every lexeme to the list of rows containing it, making @@ to_tsquery(...) queries very fast.

```
CREATE INDEX ON guards_profile USING gin (search_vector);
```

BRIN (Block Range Index) — Time-Series

Used for recorded_at / created_at on append-only tables where rows are physically inserted in timestamp order. A BRIN index stores only the min/max value per block range (default 128 pages). It is tiny (kilobytes vs gigabytes for B-tree) and sufficient for coarse time-range scans on very large tables.

```
CREATE INDEX ON tracking_location_snapshot  
    USING brin (recorded_at) WITH (pages_per_range = 128);
```

When to use BRIN vs B-tree on timestamps: Use BRIN when (a) the table is append-only, (b) rows are inserted in roughly timestamp order, and (c) queries filter on wide time ranges (hours/days). Use B-tree when queries filter on narrow ranges (seconds/minutes) or when you need ORDER BY recorded_at LIMIT N pushdown.

3. Indexes by Table

users_user

```
-- Unique index on phone_number – the primary login identifier.  
-- Unique constraint creates an implicit B-tree index, but we make  
    it explicit  
-- to control the name and allow CONCURRENTLY creation.  
CREATE UNIQUE INDEX CONCURRENTLY IF NOT EXISTS  
    idx_users_user_phone  
    ON users_user (phone_number);  
  
-- Partial unique index on email – email is optional (nullable),  
    and  
-- a standard UNIQUE constraint would allow only one NULL, which  
    is wrong.  
-- This partial index enforces uniqueness only among non-null  
    emails.  
CREATE UNIQUE INDEX CONCURRENTLY IF NOT EXISTS  
    idx_users_user_email_notnull  
    ON users_user (email)  
    WHERE email IS NOT NULL;
```

```

-- Partial index on active users – most application queries filter
  on
-- is_active = true. This index is ~85% smaller than a full index
  if
-- 85% of users are active, and perfectly covers list/search
  queries.
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_users_user_active
  ON users_user (id)
  WHERE is_active = true;

```

users_otp

```

-- Composite partial index for OTP lookup – the canonical OTP
  validation
-- query is: WHERE phone_number = $1 AND otp_type = $2 AND is_used
  = false
-- AND expires_at > now(). This index covers the first three
  predicates;
-- expires_at is checked as a filter after index scan.
-- Partial WHERE is_used = false keeps the index tiny: only unused
  OTPs matter.
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_users_otp_lookup
  ON users_otp (phone_number, otp_type, is_used)
  WHERE is_used = false;

-- Partial index on expires_at for the background expiry cleanup
  job.
-- Only indexes future/recently-expired OTPs; old records are
  irrelevant.
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_users_otp_expires_at
  ON users_otp (expires_at)
  WHERE is_used = false;

```

guards_profile

```

-- GiST spatial index on current_location (GEOGRAPHY type).
-- Required for ST_DWithin radius queries and KNN (<=>) distance
  ordering.
-- Without this index, every "find guards near me" query is a full
  table scan.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
  idx_guards_profile_location
  ON guards_profile USING gist (current_location);

-- Partial index for available + online guards – the most frequent
  read path.
-- Only a fraction of guards are simultaneously available and
  online.
-- This index powers the real-time guard discovery feed.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
  idx_guards_profile_available_online

```

```

    ON guards_profile (id)
    WHERE is_available = true AND is_online = true;

-- Index on verification_status – used by admin dashboards and
-- onboarding
-- workflows to filter guards pending document review.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_guards_profile_verification_status
    ON guards_profile (verification_status);

-- Index on tier – used for tier-based pricing and filtered
-- discovery
-- (e.g., "show only Elite guards").
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_guards_profile_tier
    ON guards_profile (tier);

-- Partial index on average_rating for guards with sufficient
-- ratings.
-- Guards with fewer than 5 reviews have unreliable ratings;
-- exclude them
-- from rating-ordered queries to avoid misleading sort order.
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_guards_profile_rating
    ON guards_profile (average_rating DESC)
    WHERE total_reviews >= 5;

-- GIN index on search_vector tsvector column.
-- Powers full-text search: "find guards named Raj in Bangalore
-- with CCTV skills".
-- Must be GIN (not GiST) for tsvector – GIN stores per-lexeme
-- posting lists.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_guards_profile_search_vector
    ON guards_profile USING gin (search_vector);

```

search_vector Population Trigger

The search_vector column must be kept in sync whenever full_name, city, or skills changes. Use a trigger function rather than application-level updates to ensure consistency even for direct DB writes (migrations, admin edits).

```

-- Trigger function: rebuilds the tsvector from three source
-- columns.
-- Weights: A = full_name (highest), B = city, C = skills
-- (lowest).
-- pg_trgm trigram index is NOT used here – tsvector handles
-- stemming and
-- stopword removal, which trigrams cannot.
CREATE OR REPLACE FUNCTION guards_profile_search_vector_update()
RETURNS trigger AS $$
BEGIN
    NEW.search_vector :=

```

```

        setweight(to_tsvector('english', coalesce(NEW.full_name,
        '')), 'A') ||
        setweight(to_tsvector('english', coalesce(NEW.city, '')),
        'B') ||
        setweight(to_tsvector('english', coalesce(NEW.skills,
        '')), 'C');
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Trigger fires on INSERT and on UPDATE of the relevant columns
-- only.
-- The WHEN condition avoids unnecessary tsvector rebuilds on
-- unrelated updates.
CREATE TRIGGER guards_profile_search_vector_trigger
    BEFORE INSERT OR UPDATE OF full_name, city, skills
    ON guards_profile
    FOR EACH ROW
    EXECUTE FUNCTION guards_profile_search_vector_update();

-- Backfill existing rows after adding the trigger:
UPDATE guards_profile SET full_name = full_name;

```

bookings_booking

```

-- User booking history – ordered by created_at DESC for timeline
-- display.
-- Composite index: equality on user_id first, then range/sort on
-- created_at.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_bookings_booking_user_created
    ON bookings_booking (user_id, created_at DESC);

-- Guard booking history – same pattern for guard's job history
-- view.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_bookings_booking_guard_created
    ON bookings_booking (guard_id, created_at DESC);

-- Status + created_at – used by admin dashboards and reporting
-- queries
-- that filter on booking status across all users/guards.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_bookings_booking_status_created
    ON bookings_booking (status, created_at DESC);

-- Partial index: pending bookings only – the matchmaking engine
-- queries
-- pending bookings every few seconds. This index is ~5% the size
-- of a
-- full status index and fits entirely in L2 cache on a modest
-- server.

```

```

CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_bookings_booking_pending
ON bookings_booking (created_at DESC)
WHERE status = 'pending';

-- Partial index: active (in-progress) bookings – queried by the
    real-time
-- tracking system. Typically <1% of all bookings at any moment.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_bookings_booking_active
ON bookings_booking (guard_id, user_id)
WHERE status = 'active';

-- Partial index: scheduled future bookings – queried by the
    scheduler
-- service to dispatch reminders and auto-assign guards.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_bookings_booking_scheduled
ON bookings_booking (scheduled_start_time ASC)
WHERE status = 'scheduled' AND scheduled_start_time IS NOT
    NULL;

-- GiST spatial index on pickup_location – used by proximity-based
-- guard assignment and "bookings near me" admin map view.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_bookings_booking_pickup_location
ON bookings_booking USING gist (pickup_location);

-- Index on payment_status – used by the payments reconciliation
    job
-- to find bookings with unpaid/failed payment status.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_bookings_booking_payment_status
ON bookings_booking (payment_status, created_at DESC);

```

bookings_bookingtimeline

```

-- Composite index: booking_id + created_at ASC.
-- Timeline events are always fetched for a specific booking in
-- chronological order. ASC matches the natural append order and
-- allows the planner to avoid a sort step.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_bookings_timeline_booking_created
ON bookings_bookingtimeline (booking_id, created_at ASC);

```

tracking_location_snapshot

Scale note: This table receives approximately 1 million rows per day (one GPS ping every ~6 seconds per active booking, across all concurrent bookings). At 50 million rows, migrate to PostgreSQL table partitioning by recorded_at month. The BRIN index remains effective within each partition.

```
-- Composite B-tree index for fetching a booking's location
  history.
-- recorded_at DESC returns the most recent location first.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_tracking_snapshot_booking_time
    ON tracking_location_snapshot (booking_id, recorded_at DESC);

-- BRIN index on recorded_at – this is the most important index on
  this table.
-- BRIN works here because rows are inserted in timestamp order
  (append-only).
-- pages_per_range=128 means one BRIN entry covers 128×8KB = 1MB
  of table data.
-- The full BRIN index for 50M rows is ~200KB vs ~1.5GB for a B-
  tree.
-- Used by cleanup jobs and reporting that query wide time ranges.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_tracking_snapshot_recorded_at_brin
    ON tracking_location_snapshot USING brin (recorded_at)
    WITH (pages_per_range = 128);

-- GiST spatial index on location – used for geofencing queries:
-- "did this guard enter/exit the designated zone during this
  booking?"
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_tracking_snapshot_location
    ON tracking_location_snapshot USING gist (location);
```

payments_wallet

```
-- Unique index: one wallet per user.
CREATE UNIQUE INDEX CONCURRENTLY IF NOT EXISTS
    idx_payments_wallet_user_id
    ON payments_wallet (user_id);

-- Partial unique index: one active wallet per guard.
-- Guards may have deactivated wallets after account suspension;
-- the partial condition ensures uniqueness only for active
  wallets.
CREATE UNIQUE INDEX CONCURRENTLY IF NOT EXISTS
    idx_payments_wallet_guard_id
    ON payments_wallet (guard_id)
    WHERE guard_id IS NOT NULL AND is_active = true;
```

payments_transaction

```
-- Wallet transaction history – the most common query in payments:
-- "show me all transactions for wallet X in descending order."
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_payments_txn_wallet_created
    ON payments_transaction (wallet_id, created_at DESC);

-- Partial index: transactions linked to a booking.
-- Not all transactions have a booking_id (e.g., wallet top-ups).
-- Partial index avoids indexing the majority of rows.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_payments_txn_booking_id
    ON payments_transaction (booking_id)
    WHERE booking_id IS NOT NULL;

-- Partial unique index on external_reference_id (Razorpay payment
    ID).
-- NULL for wallet top-ups without an external reference.
CREATE UNIQUE INDEX CONCURRENTLY IF NOT EXISTS
    idx_payments_txn_external_ref
    ON payments_transaction (external_reference_id)
    WHERE external_reference_id IS NOT NULL;

-- Partial index: failed/pending transactions – queried by the
    reconciliation
-- job that retries or escalates stalled payments.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_payments_txn_status_pending
    ON payments_transaction (created_at DESC)
    WHERE status IN ('pending', 'failed');
```

payments_razorpayorder

```
-- Unique index on razorpay_order_id – used on every Razorpay
    webhook
-- to find the corresponding order record. Must be unique and
    fast.
CREATE UNIQUE INDEX CONCURRENTLY IF NOT EXISTS
    idx_payments_rzp_order_id
    ON payments_razorpayorder (razorpay_order_id);

-- Index on wallet_id – used to look up Razorpay orders for a
    wallet
-- (e.g., retry flow, order history).
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_payments_rzp_wallet_id
    ON payments_razorpayorder (wallet_id);
```

payments_payout

```
-- Guard payout history – ordered by created_at DESC.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_payments_payout_guard_created
ON payments_payout (guard_id, created_at DESC);

-- Partial index: payouts that haven't been processed yet.
-- Queried by the payout scheduler every minute.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_payments_payout_status_pending
ON payments_payout (created_at ASC)
WHERE status IN ('pending', 'processing');
```

reviews_review

```
-- Unique index: one review per booking – enforces the business
rule
-- that a booking can be reviewed exactly once.
CREATE UNIQUE INDEX CONCURRENTLY IF NOT EXISTS
    idx_reviews_review_booking_id
ON reviews_review (booking_id);

-- Guard review feed – all reviews for a guard, newest first.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_reviews_review_guard_created
ON reviews_review (reviewed_guard_id, created_at DESC);

-- Reviews written by a user – for "my reviews" profile section.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_reviews_review_reviewer_user
ON reviews_review (reviewer_user_id);

-- Rating distribution index – used for computing rating
histograms
-- and filtering guards by minimum rating threshold.
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_reviews_review_rating
ON reviews_review (rating, reviewed_guard_id);
```

safety_rosevent

```
-- FK index on booking_id – SOS events are fetched alongside
booking data.
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_safety_sos_booking_id
ON safety_rosevent (booking_id);

-- User SOS history – ordered by triggered_at DESC for incident
timeline.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_safety_sos_user_triggered
```

```

    ON safety_rosevent (user_id, triggered_at DESC);

-- Partial index: unresolved SOS events – the emergency response
  dashboard
-- queries ONLY unresolved events in real time. This index is tiny
  (ideally
-- zero rows, i.e., no active emergencies) and always hot in
  buffer cache.
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_safety_sos_unresolved
  ON safety_rosevent (triggered_at DESC)
  WHERE resolved_at IS NULL;

```

safety_checkin

```

-- Check-in schedule for a booking – fetched in scheduled_at order
-- to determine the next required check-in.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
  idx_safety_checkin_booking_scheduled
  ON safety_checkin (booking_id, scheduled_at ASC);

-- Partial index: overdue check-ins – the safety monitor queries
  this
-- every 30 seconds to detect guards who missed a check-in.
-- "Overdue" = status is pending AND scheduled time has passed.
-- This is a time-sensitive query on a typically tiny result set.
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_safety_checkin_overdue
  ON safety_checkin (scheduled_at ASC)
  WHERE status = 'pending' AND scheduled_at < now();

```

Note on the overdue index: `scheduled_at < now()` is a dynamic expression. PostgreSQL evaluates the WHERE clause at query time, not at index creation time — the partial index correctly shrinks as overdue check-ins are resolved. However, VACUUM must run regularly to remove dead tuples from this index as rows transition out of the partial condition.

notifications_notification

```

-- User notification feed – ordered by created_at DESC for inbox
  display.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
  idx_notifications_recipient_created
  ON notifications_notification (recipient_user_id, created_at
    DESC);

-- Partial index: unread notifications – the notification badge
  counter
-- queries this constantly. Typically ~20% of total notifications.
-- This index powers the unread count query and the unread inbox
  view.

```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_notifications_unread
ON notifications_notification (recipient_user_id, created_at
DESC)
WHERE is_read = false;
```

guards_document

```
-- FK index on guard_id – document list is always fetched per
guard.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
idx_guards_document_guard_id
ON guards_document (guard_id);

-- Partial index: documents pending verification – queried by the
-- admin document review queue. Only a subset of all documents.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
idx_guards_document_pending
ON guards_document (uploaded_at ASC)
WHERE verification_status = 'pending';
```

guards_availability

```
-- Guard weekly availability schedule – always queried by guard +
day.
-- Used during booking creation to validate guard availability on
a given day.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
idx_guards_availability_guard_day
ON guards_availability (guard_id, day_of_week);
```

admin_auditlog

```
-- Audit log by actor – "what did admin user X do?" query.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
idx_admin_auditlog_actor_time
ON admin_auditlog (performed_by_id, performed_at DESC);

-- Object audit history – "what happened to booking #123?" query.
-- content_type_id identifies the model (e.g., Booking), object_id
is the PK.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
idx_admin_auditlog_object_time
ON admin_auditlog (content_type_id, object_id, performed_at
DESC);
```

4. Partial Indexes

A partial index is a B-tree (or GiST/GIN) index built only over rows satisfying a WHERE predicate. The PostgreSQL planner will use a partial index whenever the query's WHERE clause implies the index predicate.

Why Partial Indexes Win on bSecure

Table	Full rows	Partial condition	Indexed rows	Size ratio
bookings_booking	10M	status = 'pending'	~500K (5%)	20× smaller
guards_profile	50K	is_available AND is_online	~7.5K (15%)	7× smaller
notifications_notification	5M	is_read = false	~1M (20%)	5× smaller
safety_rosevent	100K	resolved_at IS NULL	~50 (<0.1%)	2000× smaller
payments_transaction	2M	status IN ('pending', 'failed')	~40K (2%)	50× smaller

A 20× smaller index means: - It fits entirely in shared_buffers (no disk I/O for repeated queries) - Index scans complete in microseconds instead of milliseconds - VACUUM and autovacuum run faster - Less write amplification on every INSERT/UPDATE

The Planner Decision

```
-- This query WILL use idx_bookings_booking_pending:
SELECT * FROM bookings_booking
WHERE status = 'pending' AND created_at > now() - interval '1
      hour';

-- This query will NOT use idx_bookings_booking_pending
-- because the WHERE clause does not imply status = 'pending':
SELECT * FROM bookings_booking
WHERE status = 'active' AND created_at > now() - interval '1
      hour';
-- → uses idx_bookings_booking_active instead
```

5. Expression Indexes

Expression indexes index the result of a function applied to a column, enabling the planner to use the index for queries that apply the same function.

```
-- Case-insensitive phone lookup (e.g., admin search normalises to
lowercase).
-- Without this index, lower(phone_number) = $1 requires a full
seq scan.
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_users_user_phone_lower
ON users_user (lower(phone_number));

-- Date-level grouping for daily reports.
-- Enables: WHERE DATE(created_at) = '2024-01-15'
-- Note: prefer range queries (created_at >= '2024-01-15' AND <
'2024-01-16')
-- for better selectivity, but this expression index supports
legacy queries.
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_bookings_booking_date
ON bookings_booking (DATE(created_at));

-- Hour-of-day analysis in IST – used by the demand forecasting
dashboard
-- to build hourly heatmaps of booking activity.
-- The AT TIME ZONE conversion must match exactly in queries.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
idx_bookings_booking_hour_ist
ON bookings_booking (
EXTRACT(HOUR FROM created_at AT TIME ZONE 'Asia/Kolkata')
);
```

Usage example:

```
-- This query uses idx_bookings_booking_hour_ist:
SELECT
EXTRACT(HOUR FROM created_at AT TIME ZONE 'Asia/Kolkata') AS
hour_ist,
count(*) AS bookings
FROM bookings_booking
WHERE created_at >= now() - interval '30 days'
GROUP BY 1
ORDER BY 1;
```

6. Covering Indexes (INCLUDE Clause)

A covering index stores extra columns in the index leaf pages (not in the B-tree structure). When all columns needed by a query are present in the index, PostgreSQL can return results without touching the heap at all — an **index-only scan**. Heap fetches drop to zero, and I/O is reduced by orders of magnitude.

Active Booking for a Guard (Index-Only Scan)

```
-- Covers: guard_id (scan key) + booking_id, status, user_id
--          (payload)
-- The "active booking" lookup is on the critical path of every
-- location
-- update and SOS event – zero heap fetches is essential here.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_bookings_booking_guard_active_covering
    ON bookings_booking (guard_id)
    INCLUDE (booking_id, status, user_id)
    WHERE status = 'active';

-- Query that achieves index-only scan:
SELECT booking_id, status, user_id
FROM bookings_booking
WHERE guard_id = $1 AND status = 'active';
-- → Index Only Scan using
--    idx_bookings_booking_guard_active_covering
-- → Heap Fetches: 0
```

Unread Notification Count (Index-Only Scan)

```
-- Covers: recipient_user_id (scan key) + id (for COUNT), is_read
--          (filter)
-- The notification badge queries this on every app open.
-- Index-only scan means COUNT(*) never touches the heap.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_notifications_unread_count_covering
    ON notifications_notification (recipient_user_id)
    INCLUDE (id, is_read)
    WHERE is_read = false;

-- Badge count query – index-only scan:
SELECT count(*)
FROM notifications_notification
WHERE recipient_user_id = $1 AND is_read = false;
-- → Aggregate → Index Only Scan
-- → Heap Fetches: 0
```

Payment History for Wallet (Index-Only Scan)

```
-- Covers: wallet_id + created_at (scan/sort keys) + amount, type,
      status (payload)
-- The payment history screen fetches the 20 most recent
  transactions.
-- With this covering index, no heap access is needed for the
  paginated list.
CREATE INDEX CONCURRENTLY IF NOT EXISTS
  idx_payments_txn_wallet_history_covering
  ON payments_transaction (wallet_id, created_at DESC)
  INCLUDE (amount, transaction_type, status);

-- Paginated payment history – index-only scan:
SELECT amount, transaction_type, status, created_at
FROM payments_transaction
WHERE wallet_id = $1
ORDER BY created_at DESC
LIMIT 20 OFFSET 0;
-- → Index Only Scan using
      idx_payments_txn_wallet_history_covering
-- → Heap Fetches: 0
```

7. GiST Spatial Indexes

How PostGIS GiST Works: The Two-Pass Approach

A PostGIS GiST index does not store exact geometries — it stores bounding boxes. A spatial query runs in two passes:

1. **Index pass (bounding box filter):** The GiST index finds all rows whose bounding box overlaps the query geometry. This is fast but may return false positives (rows whose bbox overlaps but the actual geometry does not).
2. **Recheck pass (exact filter):** PostgreSQL fetches each candidate row from the heap and applies the exact spatial predicate (e.g., ST_DWithin) to eliminate false positives.

For GEOGRAPHY columns (used in bSecure for real-world distance calculations), ST_DWithin performs the exact geodesic distance check in the recheck pass.

KNN Query: Finding Nearest Guards

```
-- Find the 10 nearest available guards to a pickup location.
-- The <-> operator performs KNN (K-Nearest Neighbour) search
  using the GiST index.
```



```
-- ORDER BY ... <=> ... LIMIT N is the canonical pattern –
    PostgreSQL uses
-- the index to retrieve results in distance order without a sort
    step.
```

SELECT

```
    gp.id,
    gp.full_name,
    gp.tier,
    gp.average_rating,
    ST_Distance(
        gp.current_location,
        ST_SetSRID(ST_MakePoint($1, $2), 4326)::geography
    ) AS distance_meters
FROM guards_profile gp
WHERE
    gp.is_available = true
    AND gp.is_online = true
    AND ST_DWithin(
        gp.current_location,
        ST_SetSRID(ST_MakePoint($1, $2), 4326)::geography,
        5000 -- 5km radius
    )
ORDER BY
    gp.current_location <=>
    ST_SetSRID(ST_MakePoint($1, $2), 4326)::geography
LIMIT 10;
```

```
-- Index usage:
-- 1. idx_guards_profile_available_online (partial) – eliminates
    offline guards
-- 2. idx_guards_profile_location (GiST) – KNN distance ordering +
    DWithin filter
```

Heatmap Query: Bookings in Bounding Box

```
-- Fetch all bookings within a map viewport bounding box (for
    admin heatmap).
-- The && operator tests bounding box overlap – this is the fast
    first pass.
-- For GEOGRAPHY columns, cast to GEOMETRY for && (bbox)
    operations.
```

SELECT

```
    b.id,
    b.status,
    ST_X(b.pickup_location::geometry) AS lng,
    ST_Y(b.pickup_location::geometry) AS lat
FROM bookings_booking b
WHERE
    b.pickup_location &&
    ST_MakeEnvelope(
        $1, $2, -- min_lng, min_lat (SW corner)
```

```

    $3, $4, -- max_lng, max_lat (NE corner)
    4326
)::geography
AND b.created_at >= now() - interval '24 hours';

-- GiST index on pickup_location eliminates rows outside the bbox
  in pass 1.
-- created_at filter applied after spatial filter reduces heap
  fetches.

```

8. Composite Index Design Decisions

Column Order Rules

The columns in a composite index must be ordered by how they appear in queries:

1. **Equality predicates first** (col = \$1) — these reduce the search space the most.
2. **Range predicates or ORDER BY columns last** (col > \$1, ORDER BY col DESC).

```

-- CORRECT: equality (user_id) then range/sort (created_at)
CREATE INDEX ON bookings_booking (user_id, created_at DESC);
-- Supports: WHERE user_id = $1 ORDER BY created_at DESC
-- Supports: WHERE user_id = $1 AND created_at > $2

-- WRONG: range column first
CREATE INDEX ON bookings_booking (created_at DESC, user_id);
-- Only supports: ORDER BY created_at DESC (no useful user_id
  filter)

```

The Low-Cardinality Status Pitfall

```

-- AVOID: standalone index on status (5 distinct values across 10M
  rows)
-- The planner will choose a seq scan because returning 2M rows
  via index
-- is slower than a sequential scan.
CREATE INDEX ON bookings_booking (status); -- ← almost never used

-- INSTEAD: combine with a high-cardinality column
CREATE INDEX ON bookings_booking (user_id, status);
-- ✓ used for "my bookings by status"

-- OR: use a partial index for a specific status value
CREATE INDEX ON bookings_booking (created_at DESC) WHERE status =
  'pending'; -- ✓

```

Don't Duplicate (a, b) and (a)

A composite index on (a, b) already supports queries that filter only on a (the planner can scan the leading columns). Creating a separate index on (a) wastes storage and write overhead.

-- You have:

```
CREATE INDEX ON bookings_booking (user_id, created_at DESC);
```

-- You do NOT need:

```
CREATE INDEX ON bookings_booking (user_id); -- ← redundant,
wastes space
```

Exception: If the single-column index would be a partial index or covering index that the composite index cannot satisfy, create it separately.

9. EXPLAIN ANALYZE Examples

Nearby Guards Query

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT gp.id, gp.full_name, gp.tier,
       gp.current_location <=> ST_SetSRID(ST_MakePoint(77.5946,
       12.9716), 4326)::geography AS dist
FROM guards_profile gp
WHERE gp.is_available = true AND gp.is_online = true
      AND ST_DWithin(gp.current_location,
                     ST_SetSRID(ST_MakePoint(77.5946, 12.9716),
                     4326)::geography, 5000)
ORDER BY dist
LIMIT 10;
```

Expected output:

```
Limit (cost=0.28..45.12 rows=10 width=64) (actual
time=1.823..2.107 rows=10 loops=1)
  Buffers: shared hit=34
  -> Index Scan using idx_guards_profile_location on
guards_profile gp
    (cost=0.28..134.56 rows=30 width=64) (actual
time=1.820..2.101 rows=10 loops=1)
    Order By: (current_location <=>
'0101000020E6100000... '::geography)
    Filter: (is_available AND is_online AND st_dwithin(...))
    Rows Removed by Filter: 3
    Buffers: shared hit=34
Planning Time: 0.312 ms
Execution Time: 2.198 ms
```

Key indicators: GiST KNN scan, no Seq Scan, execution time < 5ms.

Booking History (User Timeline)

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT id, status, created_at, guard_id
FROM bookings_booking
WHERE user_id = 42
ORDER BY created_at DESC
LIMIT 20;
```

Expected output:

```
Limit  (cost=0.56..8.91 rows=20 width=32) (actual
time=0.043..0.089 rows=20 loops=1)
  Buffers: shared hit=4
  -> Index Scan using idx_bookings_booking_user_created on
bookings_booking
      (cost=0.56..241.34 rows=578 width=32) (actual
time=0.041..0.082 rows=20 loops=1)
      Index Cond: (user_id = 42)
      Buffers: shared hit=4
Planning Time: 0.085 ms
Execution Time: 0.103 ms
```

Unread Notification Count (Index-Only Scan)

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT count(*)
FROM notifications_notification
WHERE recipient_user_id = 42 AND is_read = false;
```

Expected output:

```
Aggregate  (cost=4.18..4.19 rows=1 width=8) (actual
time=0.031..0.031 rows=1 loops=1)
  Buffers: shared hit=3
  -> Index Only Scan using
idx_notifications_unread_count_covering
      on notifications_notification
      (cost=0.56..4.05 rows=52 width=0) (actual
time=0.018..0.025 rows=52 loops=1)
      Index Cond: (recipient_user_id = 42)
      Filter: (NOT is_read)
      Heap Fetches: 0          ← zero heap access: pure index-
only scan
      Buffers: shared hit=3
Planning Time: 0.091 ms
Execution Time: 0.048 ms
```

Heap Fetches: 0 confirms the covering index is working correctly.

Overdue Check-In Detection

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT sc.id, sc.booking_id, sc.scheduled_at
FROM safety_checkin sc
WHERE sc.status = 'pending' AND sc.scheduled_at < now()
ORDER BY sc.scheduled_at ASC;
```

Expected output:

```
Index Scan using idx_safety_checkin_overdue on safety_checkin sc
(cost=0.15..12.34 rows=3 width=24) (actual time=0.019..0.024
rows=3 loops=1)
  Buffers: shared hit=2
Planning Time: 0.074 ms
Execution Time: 0.038 ms
```

The partial index contains only (status='pending' AND scheduled_at < now()) rows; the scan is near-instantaneous regardless of total table size.

10. Index Maintenance

Creating Indexes Safely in Production

```
-- ALWAYS use CONCURRENTLY to avoid blocking writes.
-- CONCURRENTLY cannot run inside a transaction block.
-- Run directly in psql or via a migration with atomic=False.
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_example
ON some_table (some_column);
```

Checking for Invalid Indexes

After a failed CREATE INDEX CONCURRENTLY (e.g., due to a unique violation or server crash), PostgreSQL leaves behind an invalid index that consumes storage but is never used by the planner.

```
-- Find all invalid indexes:
SELECT
    schemaname,
    tablename,
    indexname,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
```

```
JOIN pg_index USING (indexrelid)
WHERE NOT indisvalid
ORDER BY pg_relation_size(indexrelid) DESC;
```

Dropping Invalid Indexes

```
-- Drop an invalid index (safe – it was never used by the
    planner):
DROP INDEX CONCURRENTLY IF EXISTS idx_example;

-- Then recreate:
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_example
    ON some_table (some_column);
```

Rebuilding a Bloated Index

```
-- REINDEX CONCURRENTLY rebuilds an index without blocking reads/
    writes.
-- Available since PostgreSQL 12. Use when pg_bloat shows >50%
    bloat.
REINDEX INDEX CONCURRENTLY idx_bookings_booking_user_created;

-- Rebuild all indexes on a table:
REINDEX TABLE CONCURRENTLY bookings_booking;
```

11. Index Bloat & Monitoring

Finding Unused Indexes

Indexes that are never scanned waste storage and slow down every INSERT/UPDATE/DELETE. Drop them.

```
-- Indexes with zero scans since last statistics reset:
SELECT
    schemaname,
    tablename,
    indexname,
    idx_scan          AS scans_since_reset,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
WHERE idx_scan = 0
    AND indexrelid NOT IN (
    -- Exclude indexes that enforce constraints (unique, PK,
    -- exclusion)
    SELECT indexrelid FROM pg_index
    WHERE indisunique OR indisprimary OR indisexclusion
```

```
)
ORDER BY pg_relation_size(indexrelid) DESC;
```

Warning: Reset `pg_stat_user_indexes` counters after a fresh deploy or vacuum. An index with `idx_scan = 0` after 7+ days of production traffic is a strong candidate for removal. Verify with `pg_stat_reset()` and re-observe.

pgstattuple Bloat Check

```
-- Install pgstattuple extension (once per database):
CREATE EXTENSION IF NOT EXISTS pgstattuple;

-- Check bloat for a specific index:
SELECT
    index_size,
    round(100 * (1 - avg_leaf_density / 100.0), 1) AS bloat_pct,
    pg_size_pretty(
        index_size::bigint * round(100 * (1 - avg_leaf_density /
        100.0), 1) / 100
    ) AS wasted_space
FROM pgstatindex('idx_bookings_booking_user_created');
```

Bloat above 30% warrants a `REINDEX CONCURRENTLY`.

Autovacuum Monitoring

```
-- Tables where autovacuum is falling behind (many dead tuples):
SELECT
    schemaname,
    relname AS tablename,
    n_dead_tup,
    n_live_tup,
    round(100.0 * n_dead_tup / nullif(n_live_tup + n_dead_tup,
    0), 1) AS dead_pct,
    last_autovacuum,
    last_autoanalyze
FROM pg_stat_user_tables
WHERE n_dead_tup > 10000
ORDER BY n_dead_tup DESC
LIMIT 20;
```

Tables with `dead_pct > 10%` and `last_autovacuum` older than 1 hour indicate autovacuum is not keeping up. Tune `autovacuum_vacuum_scale_factor` for high-write tables like `tracking_location_snapshot`.

Index Size Summary

-- All indexes sorted by size:

```
SELECT
    schemaname,
    tablename,
    indexname,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size,
    idx_scan,
    idx_tup_read,
    idx_tup_fetch
FROM pg_stat_user_indexes
ORDER BY pg_relation_size(indexrelid) DESC;
```

12. Full Index Creation Script

This is the canonical idempotent script. Run it after a fresh database setup or to ensure all indexes exist. All statements use `CONCURRENTLY IF NOT EXISTS` and must be executed **outside a transaction block**.

```
-- =====
-- bSecure PostgreSQL Index Creation Script
-- Run with: psql $DATABASE_URL -f indexes.sql
-- Must be run OUTSIDE a transaction (no BEGIN/COMMIT wrapper)
-- =====

-- -----
-- users_user
-- -----

CREATE UNIQUE INDEX CONCURRENTLY IF NOT EXISTS
    idx_users_user_phone
ON users_user (phone_number);

CREATE UNIQUE INDEX CONCURRENTLY IF NOT EXISTS
    idx_users_user_email_notnull
ON users_user (email)
WHERE email IS NOT NULL;

CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_users_user_active
ON users_user (id)
WHERE is_active = true;

-- -----
-- users_otp
-- -----

CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_users_otp_lookup
ON users_otp (phone_number, otp_type, is_used)
WHERE is_used = false;

CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_users_otp_expires_at
```



```
ON users_otp (expires_at)
WHERE is_used = false;
```

```
-- guards_profile
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_guards_profile_location
ON guards_profile USING gist (current_location);
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_guards_profile_available_online
ON guards_profile (id)
WHERE is_available = true AND is_online = true;
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_guards_profile_verification_status
ON guards_profile (verification_status);
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_guards_profile_tier
ON guards_profile (tier);
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_guards_profile_rating
ON guards_profile (average_rating DESC)
WHERE total_reviews >= 5;
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_guards_profile_search_vector
ON guards_profile USING gin (search_vector);
```

```
-- bookings_booking
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_bookings_booking_user_created
ON bookings_booking (user_id, created_at DESC);
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_bookings_booking_guard_created
ON bookings_booking (guard_id, created_at DESC);
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_bookings_booking_status_created
ON bookings_booking (status, created_at DESC);
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_bookings_booking_pending
ON bookings_booking (created_at DESC)
WHERE status = 'pending';
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_bookings_booking_active
ON bookings_booking (guard_id, user_id)
WHERE status = 'active';
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_bookings_booking_scheduled
ON bookings_booking (scheduled_start_time ASC)
WHERE status = 'scheduled' AND scheduled_start_time IS NOT
    NULL;
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_bookings_booking_pickup_location
ON bookings_booking USING gist (pickup_location);
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_bookings_booking_payment_status
ON bookings_booking (payment_status, created_at DESC);
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_bookings_booking_guard_active_covering
ON bookings_booking (guard_id)
INCLUDE (booking_id, status, user_id)
WHERE status = 'active';
```

```
-- bookings_bookingtimeline
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_bookings_timeline_booking_created
ON bookings_bookingtimeline (booking_id, created_at ASC);
```

```
-- tracking_location_snapshot
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_tracking_snapshot_booking_time
ON tracking_location_snapshot (booking_id, recorded_at DESC);
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_tracking_snapshot_recorded_at_brin
ON tracking_location_snapshot USING brin (recorded_at)
WITH (pages_per_range = 128);
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_tracking_snapshot_location
ON tracking_location_snapshot USING gist (location);
```

```
-- payments_wallet
```

```
CREATE UNIQUE INDEX CONCURRENTLY IF NOT EXISTS
    idx_payments_wallet_user_id
ON payments_wallet (user_id);
```

```
CREATE UNIQUE INDEX CONCURRENTLY IF NOT EXISTS
    idx_payments_wallet_guard_id
ON payments_wallet (guard_id)
```

```
WHERE guard_id IS NOT NULL AND is_active = true;
```

```
-- payments_transaction
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_payments_txn_wallet_created
    ON payments_transaction (wallet_id, created_at DESC);
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_payments_txn_booking_id
    ON payments_transaction (booking_id)
    WHERE booking_id IS NOT NULL;
```

```
CREATE UNIQUE INDEX CONCURRENTLY IF NOT EXISTS
    idx_payments_txn_external_ref
    ON payments_transaction (external_reference_id)
    WHERE external_reference_id IS NOT NULL;
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_payments_txn_status_pending
    ON payments_transaction (created_at DESC)
    WHERE status IN ('pending', 'failed');
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_payments_txn_wallet_history_covering
    ON payments_transaction (wallet_id, created_at DESC)
    INCLUDE (amount, transaction_type, status);
```

```
-- payments_razorpayorder
```

```
CREATE UNIQUE INDEX CONCURRENTLY IF NOT EXISTS
    idx_payments_rzp_order_id
    ON payments_razorpayorder (razorpay_order_id);
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_payments_rzp_wallet_id
    ON payments_razorpayorder (wallet_id);
```

```
-- payments_payout
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_payments_payout_guard_created
    ON payments_payout (guard_id, created_at DESC);
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_payments_payout_status_pending
    ON payments_payout (created_at ASC)
    WHERE status IN ('pending', 'processing');
```

```
-- reviews_review
```

```
CREATE UNIQUE INDEX CONCURRENTLY IF NOT EXISTS
    idx_reviews_review_booking_id
ON reviews_review (booking_id);

CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_reviews_review_guard_created
ON reviews_review (reviewed_guard_id, created_at DESC);

CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_reviews_review_reviewer_user
ON reviews_review (reviewer_user_id);

CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_reviews_review_rating
ON reviews_review (rating, reviewed_guard_id);
```

```
-- safety_osevent
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_safety_sos_booking_id
ON safety_osevent (booking_id);

CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_safety_sos_user_triggered
ON safety_osevent (user_id, triggered_at DESC);

CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_safety_sos_unresolved
ON safety_osevent (triggered_at DESC)
WHERE resolved_at IS NULL;
```

```
-- safety_checkin
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_safety_checkin_booking_scheduled
ON safety_checkin (booking_id, scheduled_at ASC);

CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_safety_checkin_overdue
ON safety_checkin (scheduled_at ASC)
WHERE status = 'pending' AND scheduled_at < now();
```

```
-- notifications_notification
```

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_notifications_recipient_created
ON notifications_notification (recipient_user_id, created_at
    DESC);

CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_notifications_unread
ON notifications_notification (recipient_user_id, created_at
    DESC)
WHERE is_read = false;
```

```

CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_notifications_unread_count_covering
ON notifications_notification (recipient_user_id)
INCLUDE (id, is_read)
WHERE is_read = false;

-- -----
-- guards_document
-- -----
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_guards_document_guard_id
ON guards_document (guard_id);

CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_guards_document_pending
ON guards_document (uploaded_at ASC)
WHERE verification_status = 'pending';

-- -----
-- guards_availability
-- -----
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_guards_availability_guard_day
ON guards_availability (guard_id, day_of_week);

-- -----
-- admin_auditlog
-- -----
CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_admin_auditlog_actor_time
ON admin_auditlog (performed_by_id, performed_at DESC);

CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_admin_auditlog_object_time
ON admin_auditlog (content_type_id, object_id, performed_at
DESC);

-- -----
-- Expression indexes
-- -----
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_users_user_phone_lower
ON users_user (lower(phone_number));

CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_bookings_booking_date
ON bookings_booking (DATE(created_at));

CREATE INDEX CONCURRENTLY IF NOT EXISTS
    idx_bookings_booking_hour_ist
ON bookings_booking (
    EXTRACT(HOUR FROM created_at AT TIME ZONE 'Asia/Kolkata')
);

-- =====

```

-- *End of index creation script*

-- =====